

トークン予算統制下における AI コードレビュー・オーケストレーション構成の欠陥検出性能比較

Comparing AI Code Review Orchestration Configurations under Matched Token Budgets

高原 大明
Hiroaki Takahara

北陸先端科学技術大学院大学
Japan Advanced Institute of Science and Technology

Keywords: LLM, code review, multi-agent systems, benchmark, token budget

Summary

Researchers and commercial tool builders have proposed multi-agent orchestration configurations for LLM-based code review, including aspect-parallel reviewers, adversarial critics, and multi-run aggregation, yet no study has compared these configurations under matched computational budgets. Current evaluations cannot show whether the orchestration structure improves detection or whether the gains come from increased computation alone. We construct a defect-detection benchmark by re-injecting real bug-fix commits from four single-language OSS repositories (Python, TypeScript, Go, and Rust) as synthesized buggy pull requests. Our pipeline mines fix commits, filters them by diff size and file count, classifies them with LLM agents, re-verifies the classifications with an adversarial pass, and stratifies the final sample of 85 bugs by category and by knowledge-cutoff timing to control for training-data leakage. One target repository is an AI-developed codebase whose entire history postdates current model cutoffs; it serves as a leakage-free control group. We describe the benchmark design and an experimental plan that compares four orchestration configurations (single reviewer, aspect-parallel, adversarial critique, and multi-run aggregation) under matched token budgets, measuring detection rate, false-positive rate, and cost efficiency.

1. はじめに

大規模言語モデル (LLM) によるコードレビューの自動化が研究・商用の両面で普及している。単一のレビューアで全観点を確認する構成に加え、観点別に並列化した複数レビューア、レビューの指摘を反証する批評エージェント、同一プロンプトの複数回実行と集約など、オーケストレーション構成の提案が相次いでいる [Tang 24]。しかし、これらの構成の優位性を報告する評価の多くは計算量 (トークン消費) を統制しておらず、観測された性能差が構成の構造に由来するのか、単なる計算量増加に由来するのかを切り分けられない。一般推論ドメインでは、予算を揃えると multi-agent debate 等の優位が消失・逆転するとの報告が複数ある [Liang 24]。偽陽性コストが非対称な欠陥検出タスクであるコードレビューにおいて、この知見が成立するかは未検証である。

本研究は次の 3 つの研究課題を設定する。 **RQ1:** トークン予算を統制したとき、オーケストレーション構成の間に欠陥検出率・偽陽性率の差があるか。 **RQ2:** 観点別並列構成の優位性 (があるとすれば) は、同一予算の multi-run 集約 (純粋な計算量増加) と比較しても残るか。 **RQ3:** 欠陥の種別によって有利な構成は異なるか。

本稿では、これらの検証に必要な、実 OSS バグを統制条件下で再注入したベンチマークの構築手法と結果、および実験計画を報告する。

2. 関連研究

Multi-agent コードレビュー。 役割分担型のレビューエージェント [Tang 24] をはじめ、multi-agent 構成による欠陥検出の提案は多い。一方、商用レビューエージェントの実証評価では、多数のツールで指摘のシグナル比が低くノイズ過多であることが報告されており、偽陽性コストを含めて評価する必要がある。

計算量統制下の multi-agent 評価。 一般推論タスクでは、multi-agent debate [Liang 24] や self-refinement [Shinn 23] の性能向上が、同一計算量の単純な多数決やベストオブ N と比較すると消失するとの報告がある。コードレビュードメインでこの統制比較を行った研究は、我々の調査の範囲では存在しない。

欠陥ベンチマーク。 Defects4J [Just 14] や SWE-bench [Jimenez 24] など実バグ由来のベンチマークが広く使われているが、いずれも修正パッチ生成の評価が主目的であり、レビュー (差分に対する指摘) の評価を目的とした、

表 1 対象リポジトリ

リポジトリ	言語	ドメイン	ライセンス
fastapi	Python	Web フレームワーク	MIT
valibot	TypeScript	スキーマ検証	MIT
chi	Go	HTTP ルータ	MIT
pgrust	Rust	DB エンジン	AGPL-3.0

計算予算を統制できる形式のベンチマークは整備されていない。また、これら既知ベンチマークは LLM の学習データに含まれる可能性が高く、暗記による性能の過大評価 (leakage) が指摘されている。

3. ベンチマーク構築

3.1 設計方針

実 OSS の bug fix コミット F を逆適用し、「バグを含む変更を提出する pull request」を合成する。基底状態を F 適用後の正常コードとし、 F^{-1} をレビュー対象の差分として提示する。これにより (1) 全リポジトリ・全バグに一律の手続きが適用でき、(2) 正解 (ground truth) が fix の変更箇所として機械的に定まる。合成 PR であって実在した PR ではない点は本手法の限界であり、考察で議論する。

3.2 対象リポジトリ

表 1 の 4 リポジトリを対象とする。リポジトリ効果と言語効果を分離するため、各リポジトリが単一言語で構成されるものを選定し、言語 (Python / TypeScript / Go / Rust) ・ドメイン (Web フレームワーク / スキーマ検証 / HTTP ルータ / データベースエンジン) を分散させた。また既存ベンチマーク (SWE-bench 等) との重複がないことを確認し、leakage リスクを下げた。

pgrust は PostgreSQL の Rust 移植であり、全履歴が 2026 年 6 月以降、すなわち現行主要モデルの知識カットオフ後に作られた AI 支援開発リポジトリである。このため (1) 完全に leakage-free な統制群として機能し、(2) AI が書いたコードを AI がレビューするという実務シナリオを代表し、(3) PostgreSQL の回帰テスト群 (46,000 クエリ超) を欠陥実在性の客観的 oracle として利用できる。一方で単独作者・AI 支援というバグ分布の偏りは threat to validity として明記する。

3.3 構築パイプライン

構築は 4 段階からなる。各段階の生成物 (候補一覧・分類判定・選定結果) はすべて JSON として公開リポジトリに保存し、第三者が選定過程を監査できるようにした。

(1) **抽出**。リポジトリごとの commit 規約に合わせた正規表現で bug fix 候補コミットを抽出した (計 1,544 件)。

(2) **機械フィルタ**。非テストコードの変更が 100 行以下かつ 5 ファイル以下、docs・typo・revert 等を除外する条件で 816 件に絞った。行数はテストコードを除いて数え

る (1 行の修正に大規模なテストが付随するケースを誤って除外しないため)。

(3) **LLM 分類と懐疑的再検証**。LLM エージェントにより各候補の diff を読ませ、(a) 実挙動バグの修正か (機能追加・リファクタ・移植作業の補完等を除外)、(b) 逆適用が自然なレビュー課題になるか、(c) バグ種別、(d) 深刻度を判定した (201 件を分類)。分類エージェントはバッチ間で判定基準がぶれるため、判定が極端なバッチには反証する側に立たせるプロンプトの再検証エージェントを適用し、4 件の判定を覆した。この結果、使用可能なバグは 136 件となった。

(4) **層別サンプリング**。乱数を用いない決定的アルゴリズム (知識カットオフ後のコミットを優先し、残りをバグ種別のラウンドロビンで補充) により、85 件 (fastapi 25 / valibot 20 / chi 15 / pgrust 25) を選定した。内訳はカットオフ後 50 件・カットオフ前 35 件であり、種別は logic 34, type-safety 15, error-handling 10, security 8, concurrency 7, その他 11 である。

3.4 Leakage 統制

使用予定モデルの知識カットオフ以降のコミットを優先的に採用し、不足分をカットオフ前から補充した上で、前後の別を共変量として分析に組み込む。これにより暗記による検出の寄与を定量化できる。事前調査では、カットオフ後の fix コミットのみで必要数を確保できるリポジトリは pgrust のみであり (他 3 リポジトリは 4~15 件)、時期混合と共変量統制は必須であった。

4. 実験計画

構築したベンチマークを用い、次の 4 構成を同一総トークン予算の下で比較する: (i) **Single**: 単一レビューが全観点を担当、(ii) **Parallel**: 観点別レビューを並列に実行し集約、(iii) **Adversarial**: レビューと、その指摘を反証する批評エージェント、(iv) **Multi-run**: 同一プロンプトを N 回実行し集約 (計算量統制の対照群)。

従属変数は欠陥検出率 (recall) ・偽陽性率・F1 およびコスト効率 (検出 1 件あたりのトークン数・金額) とし、独立変数はレビュー構成 4 種 × モデル 2-3 種 × トークン予算 2-3 水準とする。検出判定は「指摘が ground truth の変更箇所と位置・論点の両方で重なるか」を LLM judge で自動判定し、サンプルの人手検証で judge の信頼性を確認する。

5. おわりに

トークン予算統制下で AI コードレビューのオーケストレーション構成を比較するためのベンチマーク構築手法を提案し、4 言語 4 リポジトリから 85 件の実バグ由来課題を構築した。今後、パイロット実験として 4 構成

× 2 モデル × 2 予算水準の比較を実施し、RQ1–RQ3 に対する予備的知見を報告する予定である。

◇ 参 考 文 献 ◇

- [Jimenez 24] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K.: SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (2024)
- [Just 14] Just, R., Jalali, D., and Ernst, M. D.: Defects4J: a database of existing faults to enable controlled testing studies for Java programs, in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ISSTA 2014, p. 437–440, New York, NY, USA (2014), Association for Computing Machinery
- [Liang 24] Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., and Tu, Z.: Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate, in Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. eds., *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17889–17904, Miami, Florida, USA (2024), Association for Computational Linguistics
- [Shinn 23] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S.: Reflexion: Language Agents with Verbal Reinforcement Learning (2023)
- [Tang 24] Tang, X., Kim, K., Song, Y., Lothritz, C., Li, B., Ezzini, S., Tian, H., Klein, J., and Bissyande, T. F.: CodeAgent: Autonomous Communicative Agents for Code Review (2024)

〔担当委員：××○○〕

19YY 年 MM 月 DD 日 受理